

Ph4nt0m 1ntrud3r

⚠️ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

A digital ghost has breached my defenses, and my sensitive data has been stolen! 😱💻 Your mission is to uncover how this phantom intruder infiltrated my system and retrieve the hidden flag.

To solve this challenge, you'll need to analyze the provided PCAP file and track down the attack method. The attacker has cleverly concealed his moves in a timely manner. Dive into the network traffic, apply the right filters and unmask the digital intruder!

Hints

- Filter your packets to narrow down your search.
 - Attacks were done in a timely manner.
 - Time is essential
-

Tools Used

- **file:** Identify PCAP file type
 - **Wireshark:** GUI tool for analyzing network traffic
 - **tshark:** Command-line Wireshark tool for packet analysis
 - **sort:** Organize timestamps in chronological order
 - **cut:** Extract specific fields from output
 - **xxd:** Convert hexadecimal to binary data
 - **base64:** Decode Base64-encoded content
 - **grep:** Text search and filtering utility
-

Complete Solution

Step 1: Download and Prepare

Download the file named `myNetworkTraffic.pcap` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file myNetworkTraffic.pcap
```

Expected output:

```
myNetworkTraffic.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4
(Raw IPv4, capture length 65535)
```

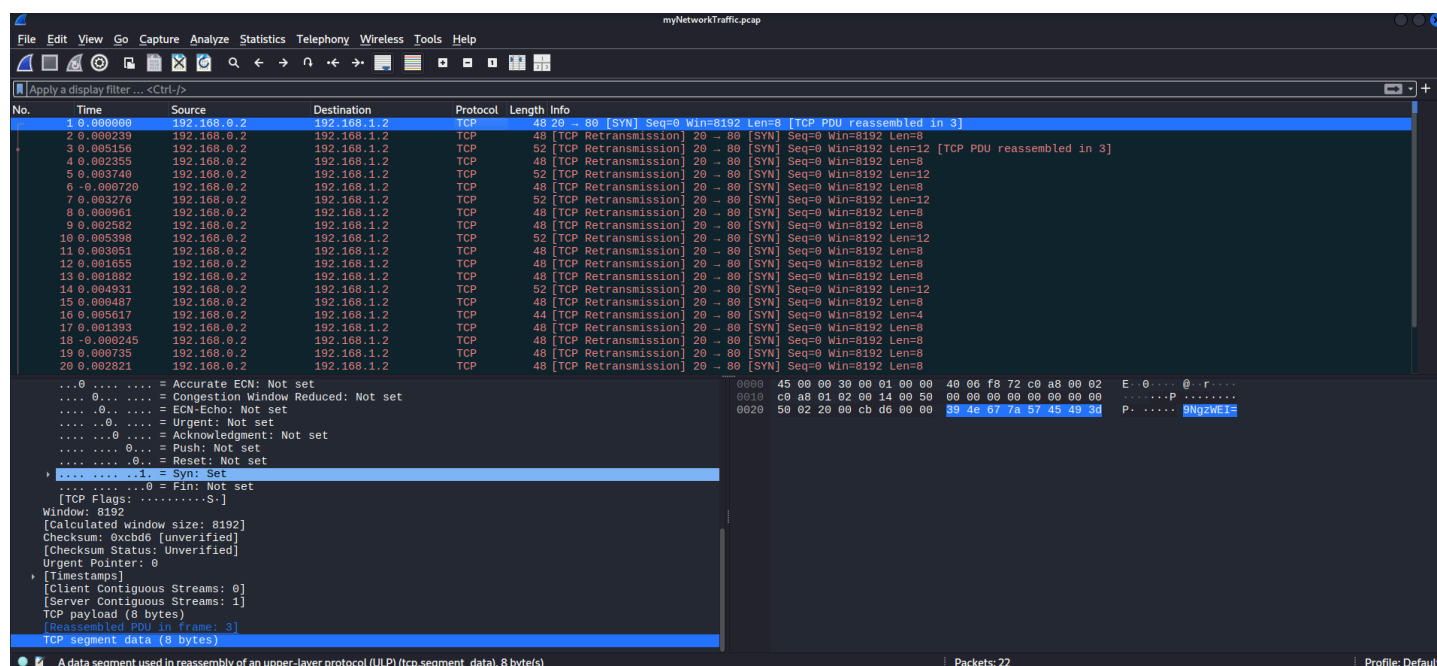
The file is a network packet capture (PCAP) that needs analysis using network forensics tools.

Step 3: Analyze with Wireshark (Visual Inspection)

Open the file in Wireshark to get a visual understanding of the traffic:

Upon inspection, observe:

- Repeated TCP packets with small payload lengths
- Most payloads contain Base64-like or hexadecimal characters
- Payload lengths are consistent: either 4 bytes or 12 bytes



Manually extracting each packet would be time-consuming, so we'll use `tshark` (command-line Wireshark) for automated extraction and processing.

Step 4: Extract All Packets with tshark

List all packets in the capture file:

```
tshark -r myNetworkTraffic.pcap
```

Output:

```
1  0.000000  192.168.0.2 → 192.168.1.2  TCP 48 20 → 80 [SYN] Seq=0 Win=8192 Len=8
2  -0.002604  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
3  -0.002141  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
4  -0.001899  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
5  0.000935  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
6  0.001804  192.168.0.2 → 192.168.1.2  TCP 52 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=12 [TCP PDU reassembled in 6]
7  0.000453  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
8  0.000676  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
9  -0.001034  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
10 0.001165  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
11 0.002819  192.168.0.2 → 192.168.1.2  TCP 52 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=12
12 0.003036  192.168.0.2 → 192.168.1.2  TCP 44 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=4
13 -0.001298  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
14 0.002367  192.168.0.2 → 192.168.1.2  TCP 52 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=12
15 -0.000720  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
16 -0.000476  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
17 0.001388  192.168.0.2 → 192.168.1.2  TCP 52 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=12
18 0.000229  192.168.0.2 → 192.168.1.2  TCP 48 [TCP Retransmission] 20 → 80 [SYN]
```

```
Seq=0 Win=8192 Len=8
19 0.002586 192.168.0.2 → 192.168.1.2 TCP 52 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=12
20 0.002045 192.168.0.2 → 192.168.1.2 TCP 52 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=12
21 -0.000228 192.168.0.2 → 192.168.1.2 TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
22 -0.001560 192.168.0.2 → 192.168.1.2 TCP 48 [TCP Retransmission] 20 → 80 [SYN]
Seq=0 Win=8192 Len=8
```

Step 5: Extract and Sort Timestamps

Use tshark to extract frame times in a structured format:

```
tshark -r myNetworkTraffic.pcap -T fields -e frame.time
```

Key tshark options explained:

- `-r myNetworkTraffic.pcap` : Read from the PCAP file
- `-T fields` : Output format as fields (tab-separated values)
- `-e frame.time` : Extract the frame timestamp field

Output:

```
2025-03-06T05:32:03.855125000+0200
2025-03-06T05:32:03.852521000+0200
2025-03-06T05:32:03.852984000+0200
2025-03-06T05:32:03.853226000+0200
2025-03-06T05:32:03.856060000+0200
2025-03-06T05:32:03.856929000+0200
2025-03-06T05:32:03.855578000+0200
2025-03-06T05:32:03.855801000+0200
2025-03-06T05:32:03.854091000+0200
2025-03-06T05:32:03.856290000+0200
2025-03-06T05:32:03.857944000+0200
2025-03-06T05:32:03.858161000+0200
2025-03-06T05:32:03.853827000+0200
2025-03-06T05:32:03.857492000+0200
2025-03-06T05:32:03.854405000+0200
2025-03-06T05:32:03.854649000+0200
2025-03-06T05:32:03.856513000+0200
2025-03-06T05:32:03.855354000+0200
2025-03-06T05:32:03.857711000+0200
```

```
2025-03-06T05:32:03.857170000+0200
2025-03-06T05:32:03.854897000+0200
2025-03-06T05:32:03.853565000+0200
```

Notice the timestamps are not in chronological order. Sort them to establish the correct sequence:

```
tshark -r myNetworkTraffic.pcap -T fields -e frame.time | sort
```

Output:

```
2025-03-06T05:32:03.852521000+0200
2025-03-06T05:32:03.852984000+0200
2025-03-06T05:32:03.853226000+0200
2025-03-06T05:32:03.853565000+0200
2025-03-06T05:32:03.853827000+0200
2025-03-06T05:32:03.854091000+0200
2025-03-06T05:32:03.854405000+0200
2025-03-06T05:32:03.854649000+0200
2025-03-06T05:32:03.854897000+0200
2025-03-06T05:32:03.855125000+0200
2025-03-06T05:32:03.855354000+0200
2025-03-06T05:32:03.855578000+0200
2025-03-06T05:32:03.855801000+0200
2025-03-06T05:32:03.856060000+0200
2025-03-06T05:32:03.856290000+0200
2025-03-06T05:32:03.856513000+0200
2025-03-06T05:32:03.856929000+0200
2025-03-06T05:32:03.857170000+0200
2025-03-06T05:32:03.857492000+0200
2025-03-06T05:32:03.857711000+0200
2025-03-06T05:32:03.857944000+0200
2025-03-06T05:32:03.858161000+0200
```

Step 6: Filter and Extract Specific Payloads

Extract only TCP packets with payloads of 4 or 12 bytes along with their timestamps:

```
tshark -r myNetworkTraffic.pcap -T fields -e frame.time -e tcp.segment_data -Y
"tcp.len==4 || tcp.len==12" | sort
```

Key tshark options explained:

- `-e tcp.segment_data` : Extract TCP payload data (in hexadecimal)

- `-Y "tcp.len==4 || tcp.len==12"` : Display filter to show only packets with payload length 4 or 12 bytes
- `| sort` : Sort output by timestamp to arrange in chronological order

Output:

```
2025-03-06T05:32:03.856513000+0200      63476c6a62304e5552673d3d
2025-03-06T05:32:03.856929000+0200      657a46305833633063773d3d
2025-03-06T05:32:03.857170000+0200      626e52666447673064413d3d
2025-03-06T05:32:03.857492000+0200      587a4d3063336c6664413d3d
2025-03-06T05:32:03.857711000+0200      596d68664e484a665a673d3d
2025-03-06T05:32:03.857944000+0200      4d7a45345a4749794d673d3d
2025-03-06T05:32:03.858161000+0200      66513d3d
```

The data is in hexadecimal format. Extract just the data portion and convert it.

Step 7: Convert Hex to ASCII and Decode

Use `cut` to extract only the hex data, then convert with `xxd` and decode with `base64` :

```
tshark -r myNetworkTraffic.pcap -T fields -e frame.time -e tcp.segment_data -Y
"tcp.len==4 || tcp.len==12" | sort | cut -f2 -d '$\t' | xxd -p -r
```

Key options explained:

- `cut -f2 -d '$\t'` : Extract field 2 (the hex data) using tab as delimiter
- `xxd -p -r` : Convert hex to binary (`-p` for plain format, `-r` for reverse)

Output:

```
cGljb0NURg==ezF0X3c0cw==bnRfdGg0dA==XzM0c3lfdA==YmhfNHJfZg==MzE4ZGIyMg==fQ==
```

This is now Base64-encoded data. Decode it to reveal the flag:

```
tshark -r myNetworkTraffic.pcap -T fields -e frame.time -e tcp.segment_data -Y
"tcp.len==4 || tcp.len==12" | sort | cut -f2 -d '$\t' | xxd -p -r | base64 -d
```

Final Result

```
picoCTF{<REDACTED>}
```

Key Concepts

- **PCAP Analysis:** Network packet captures contain timestamped records of all traffic, useful for forensic investigations.
- **Packet Filtering:** Using display filters (`-Y` option) to focus on specific traffic patterns.
- **Payload Extraction:** Recovering data hidden in TCP payloads using field extraction and hex conversion.
- **Data Reconstruction:** Combining multiple small payloads by sorting timestamps chronologically.
- **Encoding Layers:** Data encoded in multiple formats (hex → binary → Base64) requires sequential decoding.
- **tshark Advantages:** Automated, scriptable packet analysis compared to manual GUI inspection.